

WEATHER-FISH

Contact person:

Fenil Ramani
f.ramani@oth-aw.de
OTH Amberg-Weiden
April 2026

1 Motivation of the Project

Weather applications are widely used, but most existing systems provide static and non-personalized information. Users receive the same weather report regardless of their preferences, context, or situation.

The existing OTH weather platform already supports basic report generation but suffers from several limitations such as delayed response time, lack of automation, and absence of intelligent personalization.

This project aims to transform the existing system into an **AI-driven adaptive weather narration system**. Instead of simply displaying weather data, the system will generate **context-aware, personalized, and human-like weather reports** using a pre-trained language model.

The focus is not only on improving performance but also on enhancing **user experience through intelligent content generation and automation**.

2 Goal

The main goal of this project is to **improve the existing OTH weather platform** by integrating AI as the core component and optimizing system performance.

Primary objectives:

- Enable **instant playback** of weather reports
- Eliminate delay using **pre-generated and cached reports**
- Integrate a **pre-trained AI model for text generation**
- Implement **context-aware personalization**
- Automate report generation using a scheduler
- Deploy the system on **Hugging Face as a standalone application**

System capabilities:

- Generate human-like weather narration
 - Adapt tone based on weather conditions and user preferences
 - Provide multiple styles (formal, casual, humorous)
 - Deliver fast and responsive interaction
-

3 Current State

The existing system provides:

- Location input (city/ZIP)
- Presenter selection
- Weather report generation
- Basic UI for interaction

Identified issues:

- Reports are generated on-demand → causes delay
 - No caching or storage mechanism
 - No automation (manual generation every time)
 - No personalization or adaptive behavior
 - Limited user engagement
-

4 Target State

The improved system will evolve into an **AI-powered intelligent weather assistant**.

Key features:

- **Instant Playback:**
Reports are pre-generated and played immediately on user request
- **Automated Generation:**
Reports are generated periodically (e.g., hourly) using a scheduler
- **AI-Based Text Generation:**
A pre-trained language model generates natural, human-like reports

- **Context-Aware Personalization:**

Tone and content adapt based on:

- Temperature
- Weather conditions
- Time of day
- User preferences

- **Audio Output (TTS):**

Generated text is converted into speech using neural TTS

- **Storage & Caching:**

Text and audio are stored for fast retrieval

- **Deployment:**

Application will be deployed on Hugging Face

5 Approach

The system will follow a **modular and AI-centric architecture**:

System Flow:

Weather API → Context Engine → Prompt Engineering → AI Model → TTS → Storage → Playback

Key Components:

1. Weather Data Layer

- Fetch real-time data using weather API
 - Extract features (temperature, condition, etc.)
-

2. Context Engine

- Analyze:
 - Weather severity
 - Time of day
 - User preferences

- Generate context parameters (e.g., warning, cheerful, advisory)
-

3. AI Text Generation

- Use pre-trained language model (e.g., LLaMA / Hugging Face)
 - Apply prompt engineering to generate adaptive reports
-

4. Audio Generation (TTS)

- Convert text to speech using edge-tts
 - Support multiple voice styles
-

5. Scheduler

- Use APScheduler for automatic report generation
 - Run periodically (hourly updates)
-

6. Storage Layer

- Store generated text and audio
 - Enable fast retrieval (caching)
-

7. Frontend

- User input (city, style)
 - Play button for instant playback
 - Display of weather information
-

6 Time Plan

- **March:** Kick-Off and team formation
Result: Project understanding and requirement analysis
- **Early April:** Planning and architecture design
Result: Final system design and task allocation

- **Mid April:** System setup and analysis
Result: Existing system executed and analyzed
 - **Late April – Early May:** Core backend improvements
Result: Storage and caching implemented
 - **Mid May:** Prototype development
Result: Basic working system with reduced delay
 - **May 29:** Milestone demonstration
Result: System with at least one improvement
 - **Early June:** Scheduler implementation
Result: Automated report generation
 - **Mid June:** AI integration
Result: Personalized AI-generated reports
 - **Late June:** TTS and UI improvements
Result: Full pipeline (AI + audio + playback)
 - **Early July:** Testing and optimization
Result: Stable and optimized system
 - **July:** Final submission and presentation
Result: Completed project with documentation
-

7 Planned Results

7.1 Milestone Result

- Functional prototype
 - At least one improvement (e.g., instant playback or TTS)
-

7.2 Final Result

- AI-powered personalized weather system
 - Instant playback capability
 - Automated report generation
 - Context-aware adaptive narration
 - Public deployment on Hugging Face
-

8 Team Members

- **Fenil Ramani (Project Lead):**
System architecture, AI integration, prompt engineering, deployment
 - **Prasad Rajyguru:**
Frontend development, UI/UX, audio playback, user interaction
 - **Shubham Kushwaha:**
Backend development, API integration, scheduler, storage
-

9 References

1. Open-Meteo API (2024) – Weather data retrieval
2. APScheduler (2024) – Task scheduling and automation
3. edge-tts (2024) – Text-to-speech conversion
4. Hugging Face Spaces (2025) – Deployment platform
5. GeoPy (2024) – Geolocation and coordinate handling
6. LLaMA (2023) – Pre-trained language model for text generation
7. AWS Caching Strategies (2024) – Performance optimization